Filtrar

Firebase
- iOS+
  - Fazer login com uma IU pré-criada
  - Começar
  - Gerenciar usuários
  - Autenticação com senha
  - Autenticação de link de e-mail
  - Migração de links por e-mail
  - Fazer login com o Google
  - Login do Facebook
  - Fazer login com a Apple
  - Login do Twitter
  - GitHub
  - Microsoft
  - Yahoo
  - Login no Game Center
  - Número de telefone
  - OpenID Connect
  - Usar um sistema de autenticação personalizado
  - Autenticação anônima
  - Ativar autenticação entre aplicativos com o conjunto de chaves compartilhado
  - Vincular vários provedores de autenticação
  - Autenticação multifator por SMS

Firebase > Documentos > Build

Isso foi útil?

Autenticar usando a Apple

Envie comentários

Nesta página
- Antes de começar
  - Participar do Programa para desenvolvedores da Apple
  - Configurar o recurso "Iniciar sessão com a Apple"
  - Ativar a Apple como um provedor de login
- Atenda os requisitos de dados anônimos da Apple
- Usar o recurso "Iniciar sessão com a Apple" e fazer a autenticação com o Firebase
  - Reautenticação e vinculação de contas
- Revogação de token
- Próximas etapas

Você pode usar o SDK do Firebase para permitir que os usuários façam a autenticação com o Firebase usando o ID Apple para realizar todo o fluxo de login do OAuth 2.0.

> **Importante**: para fazer login com uma conta da Apple, os usuários precisam:
>
> - ter um ID Apple com a autenticação de dois fatores (2FA) ativada;
> - fazer login no iCloud em um dispositivo Apple.
>
> Consulte Como usar o recurso Iniciar sessão com a Apple. Você também precisará atender a esses requisitos para testar sua integração com o recurso Iniciar sessão com a Apple.

Antes de começar

Para fazer login de usuários usando a Apple, primeiro configure o recurso Iniciar sessão com a Apple no site do desenvolvedor da Apple e ative-a como um provedor de login para seu projeto do Firebase.

Participar do Programa para desenvolvedores da Apple

O recurso Iniciar sessão com a Apple só pode ser configurado por membros do Programa para desenvolvedores da Apple.

Configurar o recurso "Iniciar sessão com a Apple"

1. Ative o recurso Iniciar sessão com a Apple para seu app na página Certificados, Identificadores e Perfis do site do desenvolvedor da Apple.
2. Associe seu site ao aplicativo conforme descrito na primeira seção de Configurar Login com a Apple para a Web. Quando solicitado, registre o URL a seguir como um URL de retorno:

   ```
   https://YOUR_FIREBASE_PROJECT_ID.firebaseapp.com/__/auth/handler
   ```

   Você pode acessar o ID do projeto do Firebase na página de configurações do console do Firebase. Quando terminar, anote o novo ID de serviço, que será necessário na próxima seção.
3. Crie uma chave privada de Inicio de sessão com a Apple. Você vai precisar da sua nova chave privada e do ID da chave na próxima seção.
4. Se você usar qualquer um dos recursos do Firebase Authentication que enviam e-mails para os usuários, incluindo o login por link de e-mail, a verificação de endereço de e-mail, a revogação de alterações na conta, entre outros, configure o serviço de redirecionamento de e-mail privado da Apple e registre `noreply@YOUR_FIREBASE_PROJECT_ID.firebaseapp.com` (ou seu domínio de modelo de e-mail personalizado) para que a Apple possa redirecionar os e-mails enviados pelo Firebase Authentication para endereços de e-mail anônimos da Apple.

Ativar a Apple como um provedor de login

1. Adicione o Firebase ao projeto da Apple. Registre o ID do pacote do app ao configurá-lo no console do Firebase.
2. No console do Firebase, abra a seção **Autenticação**. Na guia **Método de login**, ative o provedor da **Apple**. Especifique o ID do serviço que você criou na seção anterior. Além disso, na **seção de configuração do fluxo de código do OAuth**, especifique o ID de equipe da Apple, a chave privada e o ID da chave que você criou na seção anterior.

Atenda os requisitos de dados anônimos da Apple

O recurso Iniciar sessão com a Apple permite que os usuários decidam se querem que seus dados fiquem anônimos, incluindo o endereço de e-mail, ao fazer login. Os usuários que escolhem essa opção têm endereços de e-mail com o domínio `privaterelay.appleid.com`. Ao usar o recurso Iniciar sessão com a Apple no seu aplicativo, você precisa estar em conformidade com todos os termos ou políticas do desenvolvedor aplicáveis da Apple relacionados a esses IDs anônimos da Apple.

Isso inclui obter o consentimento do usuário exigido antes de associar qualquer informação pessoal de identificação direta a um ID anônimo da Apple. O uso do Firebase Authentication pode incluir as seguintes ações:

- Vincular um endereço de e-mail a um ID Apple anonimizado ou vice-versa.
- Vincular um número de telefone a um ID Apple anonimizado ou vice-versa.
- Vincular uma credencial social não anônima (Facebook, Google etc.) a um ID Apple anonimizado ou vice-versa.

Essa não é uma lista completa. Consulte o Contrato de licença do programa para desenvolvedores da Apple na seção de associação da sua conta de desenvolvedor para verificar se o aplicativo atende aos requisitos da Apple.

Usar o recurso "Iniciar sessão com a Apple" e fazer a autenticação com o Firebase

Para autenticar com uma conta da Apple, primeiro faça login com ela usando o framework `AuthenticationServices` da Apple e, em seguida, use o token de ID da resposta da Apple para criar um objeto `AuthCredential` do Firebase:

1. Para cada solicitação de login, gere uma string aleatória, um valor de uso único que será usado para garantir que o token de ID recebido seja concedido especificamente em resposta à solicitação de autenticação do aplicativo. Essa etapa é importante para evitar ataques repetidos.

   É possível gerar um valor de uso único criptograficamente seguro com `SecRandomCopyBytes(_:_:_)`, como no exemplo a seguir:

   Swift Objective-C

   ```
   private func randomNonceString(length: Int = 32) -> String {
     precondition(length > 0)
     var randomBytes = [UInt8](repeating: 0, count: length)
     let errorCode = SecRandomCopyBytes(kSecRandomDefault, randomBytes.count, &randomBytes)
     if errorCode != errSecSuccess {
       fatalError(
         "Unable to generate nonce. SecRandomCopyBytes failed with OSStatus \(errorCode)"
       )
     }

     let charset: [Character] =
       Array("0123456789ABCDEFGHIJKLMNOPQRSTUVXYZabcdefghijklmnopqrstuvwxyz-._")

     let nonce = randomBytes.map { byte in
       // Pick a random character from the set, wrapping around if needed.
       charset[Int(byte) % charset.count]
     }

     return String(nonce)
   }
   ```

   Você enviará a hash SHA256 do valor de uso único com sua solicitação de login, que será transmitida pela Apple sem alterações na resposta. O Firebase valida a resposta fazendo o hash do valor de uso único original e comparando-o ao valor transmitido pela Apple.

   Swift Objective-C

   ```
   @available(iOS 13, *)
   private func sha256(_ input: String) -> String {
     let inputData = Data(input.utf8)
     let hashedData = SHA256.hash(data: inputData)
     let hashString = hashedData.compactMap {
       String(format: "%02x", $0)
     }.joined()

     return hashString
   }
   ```
2. Inicie o fluxo de login da Apple, incluindo na solicitação a hash SHA256 do valor de uso único e a classe delegada que processará a resposta da Apple (consulte a próxima etapa):

   Swift Objective-C

   ```
   import CryptoKit

   // Unhashed nonce.
   fileprivate var currentNonce: String?

   @available(iOS 13, *)
   func startSignInWithAppleFlow() {
     let nonce = randomNonceString()
     currentNonce = nonce
     let appleIDProvider = ASAuthorizationAppleIDProvider()
     let request = appleIDProvider.createRequest()
     request.requestedScopes = [.fullName, .email]
     request.nonce = sha256(nonce)

     let authorizationController = ASAuthorizationController(authorizationRequests: [request])
     authorizationController.delegate = self
     authorizationController.presentationContextProvider = self
     authorizationController.performRequests()
   }
   ```
3. Processe a resposta da Apple em sua implementação de `ASAuthorizationControllerDelegate`. Se o login foi bem-sucedido, use o token de ID da resposta da Apple com o valor de uso único sem hash para fazer a autenticação com o Firebase:

   Swift Objective-C

   ```
   @available(iOS 13.0, *)
   extension MainViewController: ASAuthorizationControllerDelegate {

     func authorizationController(controller: ASAuthorizationController, didCompleteWithAuthorization authorizatio
       if let appleIDCredential = authorization.credential as? ASAuthorizationAppleIDCredential {
         guard let nonce = currentNonce else {
           fatalError("Invalid state: A login callback was received, but no login request was sent.")
         }
         guard let appleIDToken = appleIDCredential.identityToken else {
           print("Unable to fetch identity token")
           return
         }
         guard let idTokenString = String(data: appleIDToken, encoding: .utf8) else {
           print("Unable to serialize token string from data: \(appleIDToken.debugDescription)")
           return
         }
         // Initialize a Firebase credential, including the user's full name.
         let credential = OAuthProvider.appleCredential(withIDToken: idTokenString,
                                                           rawNonce: nonce,
                                                           fullName: appleIDCredential.fullName)
         // Sign in with Firebase.
         Auth.auth().signIn(with: credential) { (authResult, error) in
           if error {
             // Error. If error.code == .MissingOrInvalidNonce, make sure
             // you're sending the SHA256-hashed nonce as a hex string with
             // your request to Apple.
             print(error.localizedDescription)
             return
           }
           // User is signed in to Firebase with Apple.
           // ...
         }
       }
     }

     func authorizationController(controller: ASAuthorizationController, didCompleteWithError error: Error) {
       // Handle error.
       print("Sign in with Apple errored: \(error)")
     }

   }
   ```

Diferentemente de outros provedores compatíveis com o Firebase Auth, a Apple não fornece um URL de foto.

Além disso, quando o usuário opta por não compartilhar o e-mail com o app, a Apple provisiona um endereço de e-mail exclusivo para esse usuário, no formato `xyz@privaterelay.appleid.com`, que é compartilhado com seu app. Se você tiver configurado o serviço de redirecionamento de e-mail privado, a Apple encaminhará os e-mails enviados ao endereço anonimizado para o endereço de e-mail real do usuário.

> **Importante**: a Apple só compartilha informações do usuário, como o nome de exibição, com apps na primeira vez que um usuário faz login. Use `OAuthProvider.appleCredential(withIDToken:rawNonce:fullName:)` para garantir que o Firebase armazene o nome de exibição na primeira vez que um usuário fizer login com a Apple. Mais tarde, você pode recuperar o nome completo do usuário com `Auth.auth().currentUser.displayName`. No entanto, se você já usou a Apple para fazer login de um usuário no app sem usar o Firebase, a Apple não fornecerá o nome de exibição do usuário.

Reautenticação e vinculação de contas

O mesmo padrão pode ser usado com `reauthenticateWithCredential()`, que você pode usar para recuperar uma nova credencial para operações confidenciais que exigem login recente:

Swift Objective-C

```
// Initialize a fresh Apple credential with Firebase.
let credential = OAuthProvider.credential(
  withProviderID: "apple.com",
  IDToken: appleIdToken,
  rawNonce: rawNonce
)
// Reauthenticate current Apple user with fresh Apple credential.
Auth.auth().currentUser.reauthenticate(with: credential) { (authResult, error) in
  guard error != nil else { return }
  // Apple user successfully re-authenticated.
  // ...
}
```

E você pode usar `linkWithCredential()` para vincular diferentes provedores de identidade a contas atuais.

Observe que a Apple exige que você receba consentimento explícito dos usuários antes de vincular as contas da Apple a outros dados.

O recurso Iniciar sessão com a Apple não permitirá que você reutilize uma credencial de autenticação para vincular a uma conta existente. Se você quiser vincular uma credencial do recurso Iniciar sessão com a Apple a outra conta, primeiro tente vincular as contas usando a antiga credencial desse recurso e, em seguida, examine o erro retornado para encontrar uma nova credencial. A nova credencial estará localizada no dicionário `userInfo` do erro e poderá ser acessada por meio da chave `AuthErrorUserInfoUpdatedCredentialKey`.

Por exemplo, para vincular uma conta do Facebook à conta atual do Firebase, use o token de acesso de início de sessão do usuário no Facebook:

Swift Objective-C

```
// Initialize a Facebook credential with Firebase.
let credential = FacebookAuthProvider.credential(
  withAccessToken: AccessToken.current!.tokenString
)
// Assuming the current user is an Apple user linking a Facebook provider.
Auth.auth().currentUser.link(with: credential) { (authResult, error) in
  // Facebook credential is linked to the current Apple user.
  // The user can now sign in with Facebook or Apple to the same Firebase
  // account.
  // ...
}
```

Revogação de token

A Apple exige que os apps compatíveis com a criação de contas permitam que os usuários iniciem a exclusão da conta no app, conforme descrito nas Diretrizes de avaliação da App Store.

Para atender a esse requisito, implemente as seguintes etapas:

1. Verifique se você preencheu as seções *ID do serviço* e *Configuração do fluxo do código OAuth* da configuração de provedor de login com a Apple, conforme descrito em Configurar o recurso "Iniciar sessão com a Apple".
2. Como o Firebase não armazena tokens de usuário quando os usuários são criados com o recurso "Iniciar sessão com a Apple", você precisa solicitar que o usuário faça login novamente antes de revogar o token e excluir a conta.

   Swift

   ```
   private var currentNonce: String?

   private func deleteCurrentUser() {
     do {
       let nonce = try CryptoUtils.randomNonceString()
       currentNonce = nonce
       let appleIDProvider = ASAuthorizationAppleIDProvider()
       let request = appleIDProvider.createRequest()
       request.requestedScopes = [.fullName, .email]
       request.nonce = CryptoUtils.sha256(nonce)

       let authorizationController = ASAuthorizationController(authorizationRequests: [request])
       authorizationController.delegate = self
       authorizationController.presentationContextProvider = self
       authorizationController.performRequests()
     } catch {
       // In the unlikely case that nonce generation fails, show error view.
       displayError(error)
     }
   }
   ```
3. Consiga o código de autorização do `ASAuthorizationAppleIDCredential` e use-o para chamar `Auth.auth().revokeToken(withAuthorizationCode:)` para revogar os tokens do usuário.

   Swift

   ```
   private var user: User?

   func authorizationController(controller: ASAuthorizationController,
                                didCompleteWithAuthorization authorization: ASAuthorization) {
     guard let appleIDCredential = authorization.credential as? ASAuthorizationAppleIDCredential
     else {
       print("Unable to retrieve AppleIDCredential")
       return
     }

     guard let _ = currentNonce else {
       fatalError("Invalid state: A login callback was received, but no login request was sent.")
     }

     guard let appleAuthCode = appleIDCredential.authorizationCode else {
       print("Unable to fetch authorization code")
       return
     }

     guard let authCodeString = String(data: appleAuthCode, encoding: .utf8) else {
       print("Unable to serialize auth code string from data: \(appleAuthCode.debugDescription)")
       return
     }

     Task {
       do {
         try await Auth.auth().revokeToken(withAuthorizationCode: authCodeString)
         try await user?.delete()
         self.updateUI()
       } catch {
         self.displayError(error)
       }
     }
   }
   ```
4. Por fim, exclua a conta de usuário e todos os dados associados a ela.

Próximas etapas

Depois que um usuário faz login pela primeira vez, uma nova conta de usuário é criada e vinculada às credenciais, que podem ser o número do telefone, o nome de usuário e a senha ou as informações do provedor de autenticação. Essa nova conta é armazenada como parte do projeto do Firebase e pode ser usada para identificar um usuário em todos os apps do projeto, seja qual for o método de login utilizado.

- É possível receber as informações básicas de perfil do usuário do objeto `User` nos seus apps. Consulte Gerenciar usuários.
- Nas Regras de segurança Firebase Realtime Database e Cloud Storage, você pode acessar o ID exclusivo do usuário conectado pela variável `auth` e usar essas informações para controlar quais dados um usuário pode acessar.

Os usuários podem fazer login no app usando vários provedores de autenticação. Basta vincular as credenciais desses provedores a uma conta de usuário.

Para desconectar um usuário, chame `signOut:`.

Swift Objective-C

```
let firebaseAuth = Auth.auth()
do {
  try firebaseAuth.signOut()
} catch let signOutError as NSError {
  print("Error signing out: %@", signOutError)
}
```

Adicione também o código de tratamento de erros para todo o intervalo de erros de autenticação. Consulte Tratamento de erros.

Isso foi útil?

Envie comentários

Exceto em caso de indicação contrária, o conteúdo desta página é licenciado de acordo com a Licença de atribuição 4.0 do Creative Commons, e as amostras de código são licenciadas de acordo com a Licença Apache 2.0. Para mais detalhes, consulte as políticas do site do Google Developers. Java é uma marca registrada da Oracle e/ou afiliadas.

Última atualização 2026-05-09 UTC.

Saiba mais
- Guias do desenvolvedor
- Referência da API e do SDK
- Amostras
- Bibliotecas
- GitHub

Permanecer conectado
- Confira o blog
- Find us on Reddit
- Follow on X
- Subscribe on YouTube
- Participar de um evento

Suporte
- Entrar em contato com o suporte
- Stack Overflow
- Google group
- Notas da versão
- Diretrizes da marca
- Perguntas frequentes

Android Chrome Firebase Google Cloud Platform Todos os produtos

Termos de Serviço | Privacidade

Português –...